

Individuelle praktische Arbeit

EcoHub Testautomation und Mapping

8. Mai 2025



Version: 0.0.4

Execution period: 06.05.2025 - 21.05.2025

Kandidat

Kento Puleo
p.kento@bluewin.ch

Berufsbildner

Sascha Listner
sascha.listner@pax.ch

Fachkraft

Kushtrim Mjeku
kushtrim.mjeku@pax.ch

Hauptexperte

Janick Wüthrich
janick.wuethrich@gmail.com

Nebenexperte

Ralf Horstmöller
ralf.horstmoeller@roche.com

DOKUMENTENVERWALTUNG

Autor: Kento Puleo
Version: 0.0.4
Datum: 8. Mai 2025
Status: In Bearbeitung
Dateiname: ipa_kp_pax_2025.pdf

Version	Datum	Änderung
0.0.1	30.04.2025	Initialisierung des $\text{\LaTeX}$ Dokuments
0.0.2	06.05.2025	Beginn der IPA
0.0.3	07.05.2025	Dokumentation der Analyse
0.0.4	08.05.2025	Dokumentation der Designphase

Inhaltsverzeichnis

I	Obligatorischer Teil	1
1	Aufgabenstellung im Originaltext	2
1.1	Titel der Arbeit	2
1.2	Ausgangslage	2
1.3	Detaillierte Aufgabenstellung	3
1.3.1	Mapping-Funktion	3
1.3.2	Testautomatisierung	3
1.4	Mittel und Methode	4
1.4.1	Entwicklungsumgebung	4
1.4.2	Versionierungsverwaltung	4
1.5	Vorkenntnisse	4
1.5.1	Frameworks und Libraries	4
1.5.2	Programmiersprachen	5
1.5.3	Versionierungsverwaltung	5
1.6	Vorarbeiten	5
1.7	Neue Lerninhalte	5
1.7.1	Unternehmen	5
1.7.2	Persönlich	5
1.8	Arbeiten in den letzten sechs Monaten	5
1.9	Individuelle Kriterien	6
2	Projektmethode	7
2.1	Interne Projektmethode	7
2.2	Gewählte Projektmethode	7
2.2.1	Die Wasserfallmethode	7
2.2.2	Die Phasen der Wasserfallmethode genauer erklärt	8
2.3	Begründung der Wahl	9
2.4	Vergleich mit agilen Methoden	9
3	Projektaufbauorganisation	11
4	Arbeitspakete	12

5 Zeitplan	15
6 Persönliches Fazit	16
7 Arbeitsjournal	17
 II Projektdokumentation	 18
8 Kurzfassung	19
8.1 Ausgangslage	19
8.2 Zielsetzung	19
8.3 Ergebnis	19
9 Initialisierung/Analyse	20
9.1 Erweiterung der Ausgangslage	20
9.1.1 IST-Stand	20
9.1.2 Darstellung der Systemstruktur im IST-Stand	22
9.1.3 Problemanalyse	25
9.1.4 Chancen und Risiken	26
9.1.5 Zielsetzung	26
9.1.6 Abgrenzung	26
9.2 Erweiterte Aufgabenstellung	26
9.3 Anforderungen	26
9.3.1 Funktionale Anforderungen	26
9.3.2 Nicht-funktionale Anforderungen	27
9.4 Rahmenbedingungen und Annahmen	28
10 Planung/Design	29
10.1 Zielbild und Anforderungen	29
10.2 Komponentendesign: Erweiterung von updateUser	30
10.3 Konfigurationsdesign: YAML-Mapping	30
10.4 Ablaufbeschreibung der Funktion updateUser	30
10.5 Testdesign und API-Abdeckung	30
10.6 Backupkonzept & Versionierung	31
10.6.1 Backupkonzept	31
10.6.2 Versionierung	31
11 Implementierung	32
11.1 Mapping-Funktionen	32
11.2 Testautomation	32

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



12 Test/Qualitätssicherung	33
12.1 Mapping-Funktionen	33
12.2 Testautomation	33
13 Ergebnisse	34
13.1 Mapping-Funktionen	34
13.2 Testautomation	34
14 Diskussion	35
14.1 Vergleiche	35
 III Anhang	 36
Hilfsmittelverzeichnis	37
Abkürzungsverzeichnis	38
Glossar	39
Abbildungsverzeichnis	40
Tabellenverzeichnis	41
Quellenverzeichnis	42

Teil I

Obligatorischer Teil

KAPITEL 1

AUFGABENSTELLUNG IM ORIGINALTEXT

Im Folgenden Kapitel wird die Detaillierte Aufgabenstellung aus dem PkOrg kopiert und formatiert, eine erweiterte Version kann im zweiten Teil der Dokumentation gefunden werden (9.1 Erweiterte Ausgangslage & 9.2 Erweiterte Aufgabenstellung).

1.1 Titel der Arbeit

Implementierung von Mapping-Funktionen und Testautomatisierung-API im EcoHub-Service

1.2 Ausgangslage

EcoHub ist eine Plattform, die die Zusammenarbeit zwischen Makler/Brokern und Versicherungen erleichtert. Seitens Pax nutzen wir dabei die Funktionen Single-Sign-On (SSO) und User-Provisioning. Makler melden sich bei EcoHub an und können über diese Plattform auf verschiedene Versicherungen zugreifen, mit denen Sie eine Service-Vereinbarung haben.

Das geschieht folgendermassen:

1. Die Pax ist als Nutzer bei EcoHub angemeldet.
2. Broker, privat oder einer Organisation, melden sich auch als Nutzer auf EcoHub an.
3. Der Broker und die Pax können eine Service-Vereinbarung treffen. Dazu müssen beide Parteien der Vereinbarung zustimmen.
4. Nach der Vereinbarung werden die Broker Daten an die Pax gesendet. Diese prüft, ob die E-Mail des Brokers gespeichert und bekannt ist.

5. Sollte die E-Mail bekannt sein, wird eine IDP-Nummer von EcoHub angefragt. Diese wird dem Benutzer zugeteilt.
6. Sollte die E-Mail nicht bekannt sein, wirft die Pax eine Fehlermeldung an EcoHub.
7. Nachdem die IDP-Nummer dem Broker zugeteilt wurde, kann dieser über die Plattform auf die Pax Seite zugreifen, ohne sich erneut anmelden zu müssen.

1.3 Detaillierte Aufgabenstellung

Im Rahmen der IPA werden zwei Aspekte des EcoHub-Services weiterentwickelt, um dessen Funktionalität und Zuverlässigkeit zu verbessern. Einerseits wird eine Mapping-Funktion implementiert, die die automatische und präzise Zuordnung von Nutzern zu Applikationsgruppen ermöglicht. Andererseits wird eine umfassende Testautomatisierung auf API-Ebene eingeführt, um die Qualität der Schnittstellen systematisch zu prüfen und Fehler frühzeitig zu erkennen.

Beide Massnahmen zielen darauf ab, den Betrieb des EcoHub-Services effizienter zu gestalten, die Wartung zu vereinfachen und eine stabile Systemumgebung zu gewährleisten. Die nachfolgenden Abschnitte erläutern die Details zu den einzelnen Implementierungen.

1.3.1 Mapping-Funktion

Die Mapping-Funktion sorgt dafür, dass neu angelegte und bestehende Nutzer ihrer Applikationsgruppen zugeordnet werden. Diese Zuordnungen werden im OpenLDAP gespeichert und enthalten die eindeutige Kennnummer jedes Nutzers. So wird eine Präzise und nachhaltige Zuordnung gewährleistet.

Testkonzept der Mapping-Funktion

Um die Funktionalität der Mapping-Funktion sicherzustellen, wird diese umfassend getestet. Dazu gehören:

- Unit-Tests, um einzelne Komponenten isoliert zu prüfen
- Component-Tests, um die Zusammenarbeit der Module zu evaluieren
- Integration-Tests, die die Funktionalität im Gesamtsystem überprüfen

Diese Teststufen gewährleisten, dass die Mapping-Funktion fehlerfrei und stabil im Produktivbetrieb läuft.

1.3.2 Testautomatisierung

API-Tests

Der EcoHub-Service umfasst vier Schnittstellen, die künftig durch Playwright-API-Tests systematisch geprüft werden. Ziel ist es, Fehler frühzeitig zu erkennen und die Zuverlässigkeit der Schnittstellen sicherzustellen. Bei

fehlschlagenden Tests wird eine Benachrichtigung an die zuständigen Personen versendet, um eine zeitnahe Problembeseitigung zu gewährleisten.

Testumgebung

Die Durchführung der Tests erfolgt sowohl in der Testumgebung innerhalb der Continuous Deployment (CD) Pipeline als auch lokal. Für die lokale Ausführung wird ein embedded LDAP eingesetzt, das eine flexible und unabhängige Testumgebung ermöglicht.

Testdaten

Um die API-Tests zu unterstützen, werden spezifische Testdaten erstellt:

- Für das lokale Testen werden die Testdaten direkt im Repository hinterlegt
- In der CD-Pipeline wird zusätzlich ein Test-User im bestehenden LDAP-System angelegt, um die Funktionalität unter möglichst realistischen Bedingungen zu prüfen

1.4 Mittel und Methode

1.4.1 Entwicklungsumgebung

- | | |
|----------------------|-------------------------|
| • Lenovo ThinkPad X1 | Kenntnisse seit 2 Jahre |
| • Windows 11 | Kenntnisse seit 3 Jahre |
| • IntelliJ Ultimate | 3 Jahre |

1.4.2 Versionierungsverwaltung

- | | |
|----------|-------------------------|
| • GitHub | Kenntnisse seit 3 Jahre |
|----------|-------------------------|

1.5 Vorkenntnisse

1.5.1 Frameworks und Libraries

- | | |
|--------------|---------------------------|
| • Playwright | Kenntnisse seit 1/2 Jahre |
| • SpringBoot | Kenntnisse seit 2 Jahre |

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



1.5.2 Programmiersprachen

- TypeScript
- Java

Kenntnisse seit 3 Jahre

Kenntnisse seit 3 Jahre

1.5.3 Versionierungsverwaltung

- GitHub

Kenntnisse seit 3 Jahre

1.6 Vorarbeiten

Um eine gute und effiziente IPA durchführen zu können, wurden gewisse Vorarbeiten geleistet, welche es ermöglichte, Wissen und Erfahrungen zu gewinnen.

- Dokument Vorlage
- Einarbeitung in die Tools

1.7 Neue Lerninhalte

1.7.1 Unternehmen

Ziel ist es, Fehler frühzeitig zu erkennen und die Zuverlässigkeit der Schnittstellen sicherzustellen. Ausserdem soll die Applikationsgruppenzuweisung zukünftige Arbeiten erleichtern und den Nutzern die gewünschten Applikationen zur Verfügung stellen.

1.7.2 Persönlich

- Der Umgang und die Erfahrung Tests auf API-Ebene zuschreiben und Automatisieren
- Repetition und Erweiterung von Java und Springboot wissen

1.8 Arbeiten in den letzten sechs Monaten

Die letzten 6 Monate wurden in verschiedene Themen investiert. Folgend die Themen die behandelt wurden und die passende Zeitspanne.

- August & September: Vergleich und PoC RxJS zu Angular Signals
- Oktober & November: Testautomatisierung Playwright E2E Pilotversuch
- Dezember: Bekanntmachung mit IPA Thema

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



- Januar & Februar: Testautomatisierung API im Brokerportal

1.9 Individuelle Kriterien

Beschreibung der individuellen Kriterien.

KAPITEL 2

PROJEKTMETHODE

In diesem Kapitel wird die Wahl der Projektmethodik für die vorliegende Arbeit dargestellt. Zunächst wird auf die intern üblicherweise eingesetzte Projektmethode eingegangen. Anschliessend wird die für dieses Projekt gewählte Vorgehensweise vorgestellt und mit alternativen Methoden verglichen. Abschliessend wird die Entscheidung für die gewählte Methode begründet.

2.1 Interne Projektmethode

In den Softwareentwicklungsteams der Pax wird auf agile Projektmethoden gesetzt. Dabei kommt eine Kombination aus Scrum und Kanban zum Einsatz. Die Projekte werden in Sprints unterteilt, die wiederum in verschiedene Phasen gegliedert sind. Auch typische Scrum-Elemente wie Reviews und Retrospektiven werden regelmäßig durchgeführt.

Zur Steuerung des Arbeitsprozesses wird jedoch ein Kanban-Board verwendet, um die Aufgaben zu visualisieren und den Fortschritt zu verfolgen. So lässt sich nachvollziehen, wer an welcher Aufgabe arbeitet und wie viel Zeit in jede einzelne Aufgabe investiert wird.

Ein Sprint dauert in der Regel etwa drei Wochen und beginnt mit einer Planungsphase. Es folgt eine Umsetzungsphase, die schließlich von einer Testphase abgelöst wird. Am Ende eines Sprints erfolgt eine Reflexion, in der Erfahrungen und Erkenntnisse gesammelt sowie dokumentiert werden.

2.2 Gewählte Projektmethode

2.2.1 Die Wasserfallmethode

Bei der Wasserfallmethode handelt es sich um ein klassisches Projektmanagement-Modell, das besonders in der Softwareentwicklung weit verbreitet ist. Sie folgt einem linearen, sequenziellen Ansatz, bei dem das

Projekt in klar abgegrenzte Phasen unterteilt wird. Jede Phase muss vollständig abgeschlossen sein, bevor die nächste beginnt, wodurch der gesamte Projektverlauf strukturiert und gut planbar ist.

Das untenstehende Diagramm des Wasserfallmodells veranschaulicht diesen Ablauf deutlich: Es zeigt, wie die Phasen Anforderungsanalyse, Planung, Umsetzung, Test und Abschluss nacheinander durchlaufen werden. In der Regel erfolgt der Fortschritt nur in eine Richtung – ähnlich wie Wasser, das einen Wasserfall hinabfließt. Dennoch kann es in Ausnahmefällen notwendig sein, zu einer vorherigen Phase zurückzukehren, etwa um entdeckte Fehler oder Fehleinschätzungen zu korrigieren.

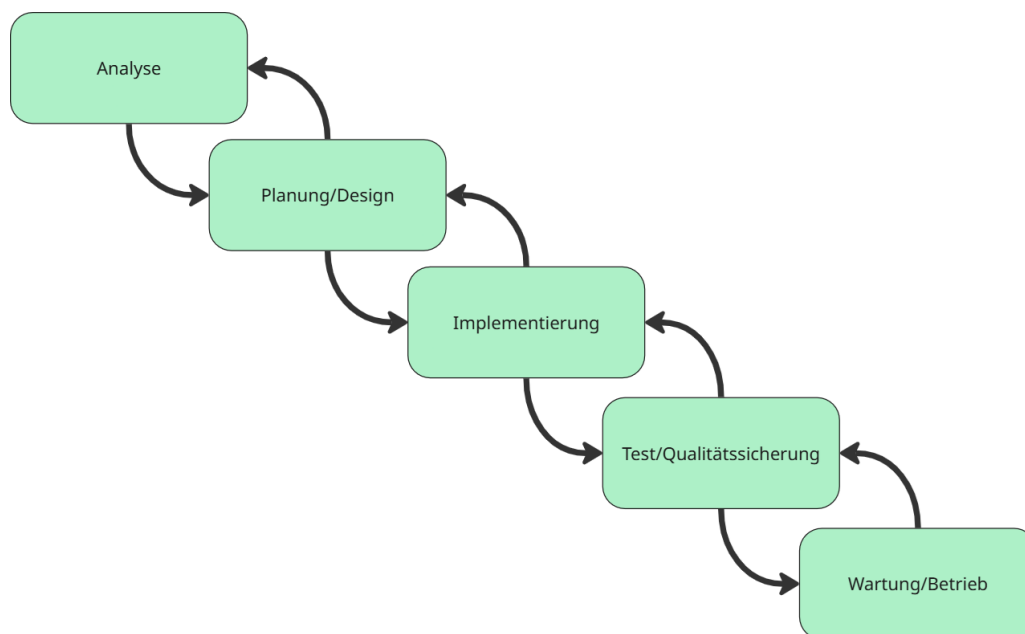


Abbildung 2.1: Wasserfallmodell (Quelle: Eigenerarbeitung)

2.2.2 Die Phasen der Wasserfallmethode genauer erklärt

Analyse

In dieser initialen Phase werden sämtliche funktionalen und nicht-funktionalen Anforderungen erhoben und dokumentiert. Gemeinsam mit allen Stakeholdern werden diese dokumentiert, der Projektumfang definiert und die Ziele festgelegt.

Planung/Design

Auf Basis der Anforderungen wird die Systemarchitektur konzipiert. Dabei werden alle Komponenten und ihre Schnittstellen definiert, die zu verwendenden Technologien und Frameworks festgelegt sowie Datenmodelle und Datenbankschemas erstellt.

Implementierung

In der Implementierungsphase erfolgt die eigentliche Programmierung der Software gemäß der Design-

Spezifikation. Die Entwickler erstellen Code nach definierten Standards, implementieren Unit-Tests und dokumentieren den Quellcode entsprechend der Vorgaben.

Test/Qualitätssicherung

Nach der Fertigstellung der einzelnen Komponenten werden diese zusammengeführt und integriert. Durch umfangreiche Tests wird die Gesamtfunktionalität validiert. Auftretende Fehler werden behoben und die Leistung des Systems optimiert.

Betrieb/Wartung

In dieser Phase wird das entwickelte System dem Kunden vorgestellt und zur abnahme bereitgestellt. Bei internen entwicklungen wird die Funktion dem Auftraggeber, meist Buissnesengineer oder Productowner, vorgestellt. Nach der Vorstellung wird die Funktion in die nächste Umgebung deployed und von den entsprechenden Personen abgenommen.

2.3 Begründung der Wahl

Die Wasserfallmethode wurde gewählt, da die Anforderungen im Vorfeld bereits klar definiert wurden und sich sehr wahrscheinlich nicht ändern werden. Es handelt sich um ein kleines Projekt, welches von einer einzigen Person ausgeführt wird, da ist es wichtig, keine komplexe Methode zu verwenden, welche womöglich viel Zeit oder Strukturierung benötigt.

2.4 Vergleich mit agilen Methoden

Für dieses Projekt gibt es keine laufenden Rückmeldungen oder Feedbackzyklen, die eine agile Methode rechtfertigen würden. Wie in der Tabelle 2.1: Vergleich zwischen Wasserfall und agilen Methoden zu sehen ist.

Tabelle 2.1: Vergleich zwischen Wasserfall und agilen Methoden

Kriterium	Wasserfall	Agile Methoden
Projektgröße	Ideal für kleine, überschaubare Projekte	Gut für große, komplexe Projekte
Anforderungsänderungen	Schwierig zu implementieren	Flexibel und einfach umsetzbar
Planung	Detaillierte Vorplanung notwendig	Iterative Planung möglich
Teamgröße	Funktioniert gut mit Einzelpersonen	Optimiert für Teams
Dokumentation	Umfangreich von Anfang an	Wächst mit dem Projekt
Risikomanagement	Risiken werden früh identifiziert	Kontinuierliche Risikobewertung

KAPITEL 3

PROJEKTAUFBAUORGANISATION

Im folgenden Kapitel wird die Projektaufbauorganisation veranschaulicht. Sie beschreibt die strukturierte Verteilung von Rollen, Aufgaben und Verantwortlichkeiten innerhalb des Projekts. Eine klar definierte Aufbauorganisation ist entscheidend für einen reibungslosen Ablauf, eine effiziente Kommunikation sowie die zielgerichtete Umsetzung der Projektziele.

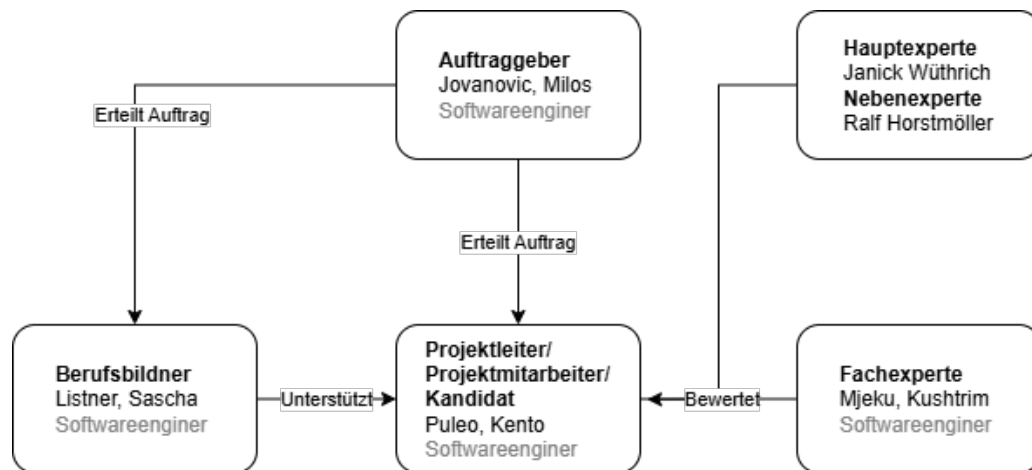


Abbildung 3.1: Projektaufbauorganisation (Quelle: Eigenerarbeitung)

In der oberen Grafik ist zu sehen welche Rolle die beteiligten Person in diesem Projekt erfüllen.

Der Auftrag wurde von einem Senior-Entwickler mit Projekterfahrung im Bereich EcoHub erteilt. Die fachliche Betreuung des Kandidaten erfolgt durch den Berufsbildner sowie einen internen Fachexperten, die ihn bereits seit etwa einem Jahr begleiten. Bei technischen Fragen steht der Berufsbildner als erste Ansprechperson zur Verfügung. Die Bewertung der Arbeit erfolgt zunächst durch den internen Fachexperten, anschließend durch einen Haupt- und einen Nebenexperten, die extern besetzt sind.

KAPITEL 4

ARBEITSPAKETE

In diesem Kapitel werden die im Projekt definierten Arbeitspakete dokumentiert. Jedes Arbeitspaket umfasst eine eindeutige ID, einen Titel, eine Beschreibung sowie die geschätzte Bearbeitungsdauer. Die Arbeitspakete dienen der strukturierten Planung und bilden die Grundlage für die Umsetzung des Projekts. Eine Übersicht aller Arbeitspakete ist in Tabelle 4.1: Aufgabenübersicht dargestellt. Da es sich um ein Einzelprojekt handelt, erfolgt keine namentliche Zuweisung der Aufgaben.

Tabelle 4.1: Aufgabenübersicht

ID	Titel	Beschreibung	Dauer
1	Definition Arbeitspakete	Es werden alle Arbeitspakete Definiert und im entsprechenden Kapitel beschrieben.	2h
2	Erfassung der Arbeitspakete im Zeitplan	Die Vorlage für den Zeitplan wird genutzt um alle Arbeitspakete zu erfassen und messbar zu machen. Es werden alle Meilensteine definiert und im Zeitplan erkennbar gemacht. Der Zeitplan wird daraufhin in die Dokumentation implementiert.	1h
3	Erarbeitung der Aufgabenstellung Originaltext	Das Kapitel Aufgabenstellung Originaltext und alle dazu gehörigen Unterkapitel werden aus dem PkOrg Kopiert und entsprechenden erweitert.	2h
4	Festlegung der Projektmethode	Es wird eine Projekt passende Projektmethode gewählt. Es wird dokumentiert weshalb diese Methode gewählt wurde und ein Vergleich mit weiteren Methoden gemacht.	2h

Fortsetzung von Tabelle 4.1

ID	Titel	Beschreibung	Dauer
5	Projektaufbauorganisation definieren und dokumentieren	Die Projektaufbauorganisation wird anhand einer Grafik und passendem Text im Dokument erfasst.	1h
6	Backupkonzept definieren und dokumentieren	Es wurde ein Backupkonzept gewählt, dokumentiert und angewandt	
7	Anforderungserhebung Rollen-Mapping	Ermittlung und Dokumentation der Anforderungen für die Mapping-Funktionalität. Es wird geklärt, welche externen Rollen existieren, wie diese intern verwendet werden sollen und welche Sonderfälle es gibt.	
8	Anforderungserhebung API-Testautomatisierung	Erhebung der Anforderungen für automatisierte API-Tests. Welche Endpunkte sollen getestet werden, welche Testarten sind relevant und wie sollen die Ergebnisse verwendet werden.	
9	Dokumentation der funktionalen und nicht-funktionalen Anforderungen	Zusammenführung und Ausarbeitung aller Anforderungen im entsprechenden Kapitel. Enthält Ziele, Systemgrenzen, funktionale und nicht-funktionale Anforderungen sowie Abgrenzungen.	
10	Priorisierung der Anforderungen	Die Anforderungen werden analysiert und die folge der Aufgaben bestimmen.	
11	Entwurf der Mapping-Architektur	Design der technischen Umsetzung für das Rollen-Mapping. Festlegung der Datenquellen, Mapping-Struktur (z.B. YAML), Verarbeitungsschritte und Integration in die bestehende Architektur.	
12	Entwurf der API-Teststruktur	Design der Testarchitektur für die API-Automatisierung. Auswahl von Tools, Aufbau der Testfälle, Verzeichnisstruktur, Wiederverwendbarkeit, Testdatenkonzept.	
13	Definition und Dokumentation der API-Testfälle	Detaillierte Ausarbeitung aller Testfälle für die zu testenden API-Endpunkte. Die Testfälle werden so beschrieben, dass sie direkt als Basis für automatisierte Tests verwendet werden können. Sie umfassen Ziel, Eingaben, erwartetes Verhalten, Vorbedingungen und Testart.	

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



Fortsetzung von Tabelle 4.1

ID	Titel	Beschreibung	Dauer
14	Grobentwurf einer Lösung erstellen	Es wurde ein Grobentwurf der beiden Lösungen erstellt und dokumentiert	
15	Implementierung der Mappingfunktion	Umsetzung der Mapping-Logik zur Zuordnung von externen Rollen auf interne LDAP-Gruppen. Implementierung wird laufend dokumentiert.	
16	Testen der Mappingfunktion	Die Mappingfunktion wurde ausgiebig getestet und die Funktionen wurden mit passenden Tests ausgestattet. Umsetzung wird laufend dokumentiert	
17	Implementierung der API Tests	Umsetzung der API Tests anhand der erstellten Testfällen. Implementierung wird laufend dokumentiert.	
18	Testen der API Tests	Die API-Tests werden auf Erfolg getestet und Fehler entsprechend behoben. Umsetzung wird laufend dokumentiert.	
19	Anbindung der Tests an CI/CD	Die API und Funktionstests sind in der Pipeline eingebunden und laufen lokal wie auch auf den Umgebungen. Umsetzung wird laufend dokumentiert.	
20	Kurzfassung des Dokuments erfassen	Die Arbeit wurde als Kurzfassung geschrieben mit 3 Kapiteln.	
21	Persönliches Fazit im Dokument festhalten	Das Persönliche Fazit wurde im Dokument erfasst.	
22	Überprüfen des Dokuments	Das Dokument wird gründlich durchgelesen und auf Fehler in der Formatierung wie aber auch in der Rechtschreibung geprüft.	

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



KAPITEL 5

ZEITPLAN

Zeitlicher Ablauf des Projekts.

KAPITEL 6

PERSÖNLICHES FAZIT

Persönliche Schlussfolgerungen und Erkenntnisse.

KAPITEL 7

ARBEITSJOURNAL

Das Arbeitsjournal dient der Nachvollziehbarkeit der Arbeiten, welche Kento Puleo zwischen dem 06.05.2025 und dem 21.05.2025 getätigt hat. Ebenso dient es zur Reflexion.

Jedes Arbeitsjournal ist gleich aufgebaut. Zuerst werden alle Tätigkeiten aufgeschrieben und Probleme oder Erfolge vermerkt. In einem weiteren Text wird eine persönliche Reflexion der Arbeit wiedergegeben. Zum Schluss wird in einer Tabelle aufgeführt, welche Arbeitspakete an diesem Tag erledigt werden konnten.

Jedes Arbeitsjournal wird vom Kandidaten (Kento Puleo) und dem Berufsbildner (Sascha Listner) unterschrieben. Das Arbeitsjournal wird täglich geführt.

Teil II

Projektdokumentation

KAPITEL 8

KURZFASSUNG

Kurze Zusammenfassung der Arbeit.

8.1 Ausgangslage

Beschreibung der Ausgangssituation.

8.2 Zielsetzung

Beschreibung der zu erreichenden Ziele.

8.3 Ergebnis

Beschreibung der erzielten Ergebnisse.

KAPITEL 9

INITIALISIERUNG/ANALYSE

In diesem Kapitel wird die aktuelle Ausgangslage analysiert, Schwachstellen identifiziert sowie die angestrebten Ziele und Anforderungen beschrieben.

9.1 Erweiterung der Ausgangslage

9.1.1 IST-Stand

Benutzer, die sich über die EcoHub-Plattform bei Pax anmelden möchten, müssen aktuell initial manuell über eine interne Eingabemaske im LDAP-Verzeichnis angelegt werden. Dabei wird zwischen Private Vorsorge (PV) und Berufliche Vorsorge (BV) unterschieden. Ein Broker, der durch diese Maske im LDAP erfasst wird, erhält anschliessend die notwendigen Rollen und Gruppen.

Dieser Prozess erfolgt manuell, ist fehleranfällig und verursacht hohen administrativen Aufwand.

Sollte sich ein Broker anmelden wollen, der nicht im LDAP vorhanden ist, scheitert der Vorgang, da aktuell keine automatische Benutzererstellung existiert. Die vorhandene *addUser*-Funktion generiert eine IDP-Nummer, welche für die Durchführung des Single Sign-On (SSO) zwingend erforderlich ist. Die Web Access Management (WAM) prüft im Rahmen des SSO auf das Vorhandensein dieser IDP-Nummer. Fehlt sie, wird der Zugriff verweigert.

Auf der EcoHub-Plattform existieren bereits eigene Berechtigungskonzepte. Diese stimmen grösstenteils mit jenen der Pax überein. Es wird zwischen Einzelleben (EL) und Kollektivleben (KL) unterschieden (siehe Abbildung 9.1). Obwohl diese Rollen über EcoHub zugewiesen werden können, werden sie derzeit nicht automatisch in das LDAP-Verzeichnis übernommen und operativ nicht weiterverwendet.

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



Einzelleben (PV) (EL_*)

Diese Rollen beziehen sich auf **Einzelverträge** der Lebensversicherung (Privatkunden).

Rollenname	Code	Beschreibung
Bestandsdaten abrufen	EL_INFO	Zugriff auf Kundendaten zu Einzelverträgen (Details, Rendite etc.).
Offertwesen	EL_OFFERT	Berechtigt zur Nutzung des Offertrechners (z. B. Erstellung neuer Angebote).
Vertragsverwaltung	EL_VERTRAG	Berechtigt zur Anpassung von Kunden- und Vertragsdaten.
Leistungsfälle	EL_LEISTUNG	Einsicht in laufende Leistungsfälle (z. B. Todesfall-Leistungen).

Kollektivleben (BV) (KL_*)

Diese Berechtigungen betreffen **Gruppenverträge**, z. B. BVG (Pensionskassen).

Rollenname	Code	Beschreibung
Bestandsdaten abrufen	KL_INFO	Zugriff auf Kundendaten zu Gruppenverträgen.
Offertwesen	KL_OFFERT	Berechtigt zur Nutzung des Offertrechners für Firmenkunden.
Vertragsverwaltung	KL_VERTRAG	Berechtigt zur Anpassung von Kollektivverträgen.
Eigene BVG-Verträge	KL_EIGENVERTRAG	Einsicht in eigene BVG-Verträge (Pensionskassenverwaltung).

Abbildung 9.1: Rollenmodell auf der EcoHub-Plattform (Quelle: Miro Board)

Business Process Model der updateUser-Funktion (BPM)

Eine *updateUser*-Funktion ist vorhanden, erfüllt jedoch noch nicht die gewünschten Anforderungen hinsichtlich automatischer Rechtezuweisung oder Synchronisierung. Wie in der Abbildung 9.2 zu sehen ist, wird bei der updateUser-Funktion die gleiche Logik wie bei der addUser-Funktion verwendet.

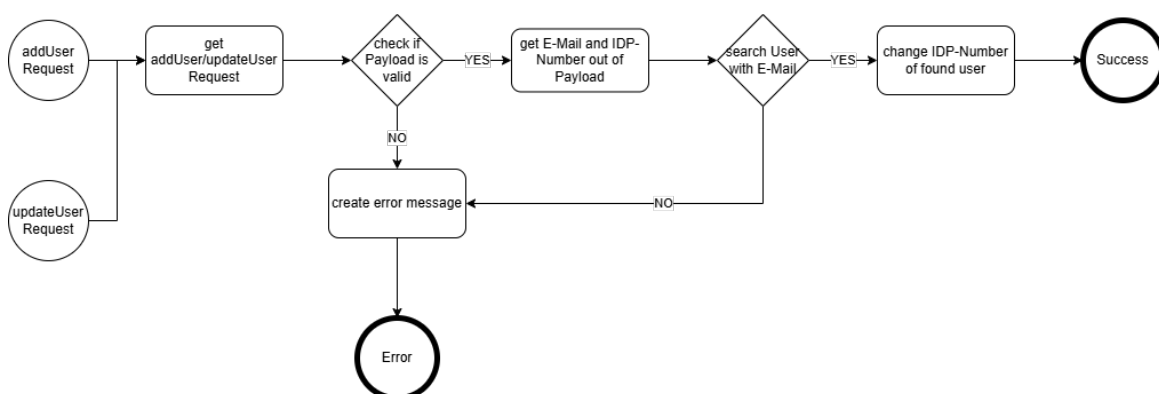


Abbildung 9.2: Business Process Model des IST-Zustands der Benutzerregistrierung (Quelle: Eigene Darstellung)

Tabelle der zu testenden Endpunkte

Hier ist eine Tabelle der zu testenden Endpunkte und deren aktuellen Funktionen zu sehen.

Tabelle 9.1: Übersicht der zu testenden Endpunkte

Endpoint	Beschreibung
addUser	Sucht einen Broker im LDAP-Verzeichnis anhand der E-Mail-Adresse und weist diesem eine IDP-Nummer zu.
updateUser	Aktualisiert die IDP-Nummer im LDAP-Verzeichnis.
deleteUser	Entfernt die IDP-Nummer eines Benutzers aus dem LDAP-Verzeichnis.
getUserStatus	Prüft die Existenz der IDP-Nummer eines Benutzers im LDAP-Verzeichnis.

9.1.2 Darstellung der Systemstruktur im IST-Stand

Folgend wird die Systemstruktur im IST-Stand dargestellt. Diese wird während dieser Arbeit jedoch nicht verändert und dient nur zur Veranschaulichung des aktuellen Zustands.

Systemkontextdiagramm (C1):

Zur Veranschaulichung des IST-Zustands wurde ein C1-Diagramm nach dem C4-Modell erstellt. Dieses stellt die Systemlandschaft im Kontext dar und zeigt die Interaktionen zwischen Benutzer, zentralem Dienst (EcoHub), dem Provisioning Service sowie den Umsystemen.

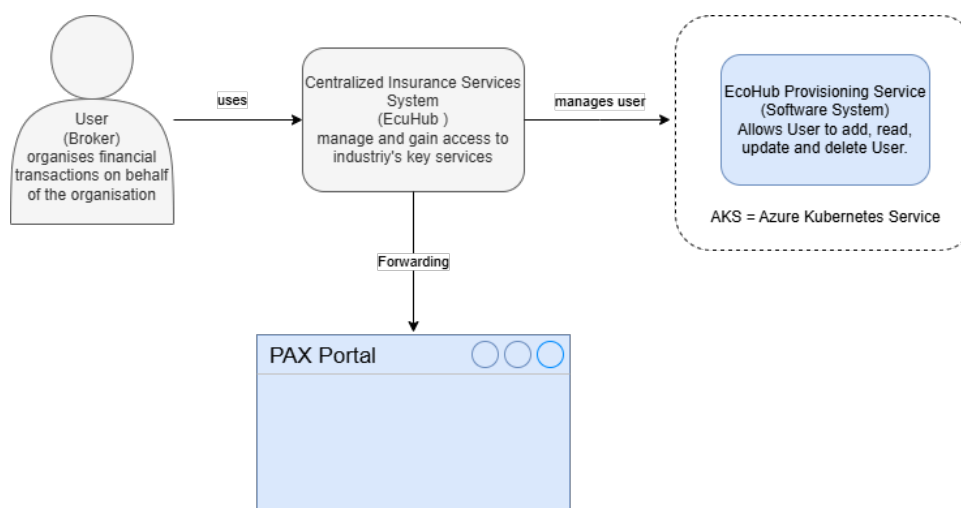


Abbildung 9.3: Systemkontext (C1) – IST-Zustand der Benutzerverwaltung (Quelle: Eigene Darstellung)

Container-Diagramm (C2)

Ergänzend zum Systemkontext (C1) wird in der folgenden Abbildung die interne Struktur des Systems im Sinne eines C2-Diagramms dargestellt. Dieses zeigt die einzelnen technischen Container, deren Hauptaufgaben sowie die Kommunikationsflüsse zwischen ihnen im aktuellen Systemzustand.

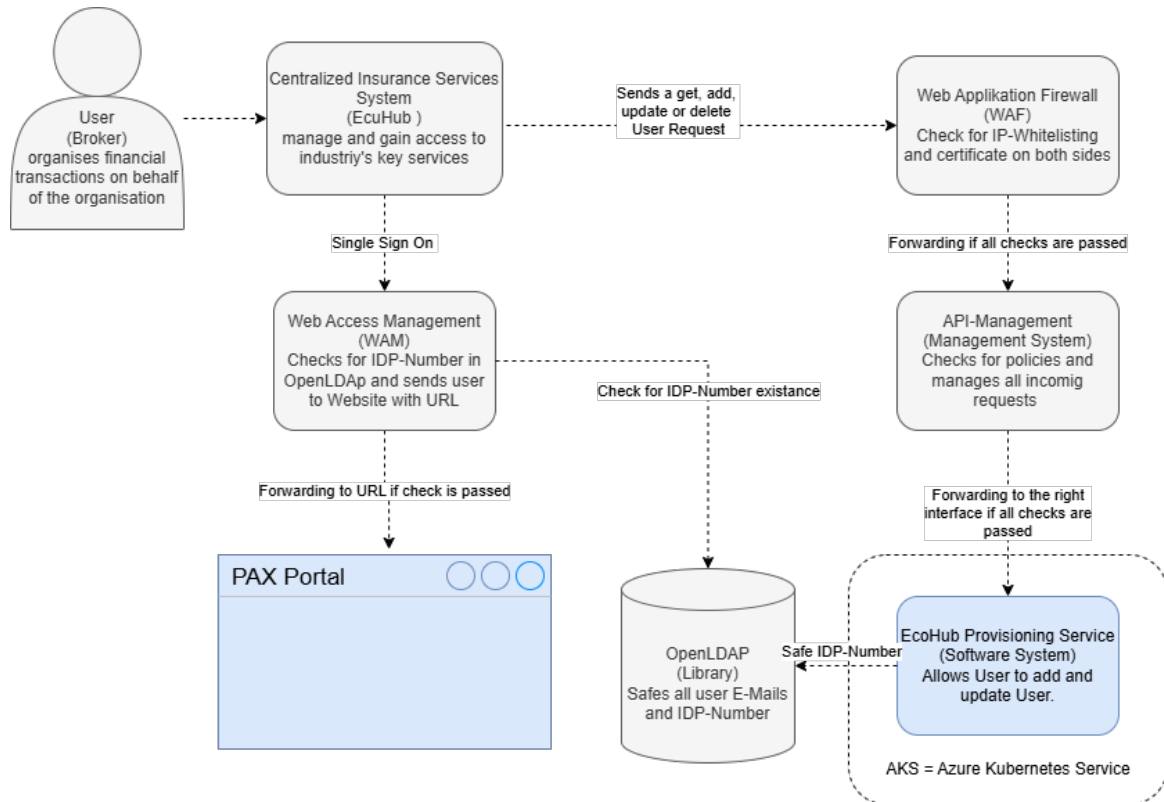


Abbildung 9.4: C2-Diagramm IST-Zustand – Containerübersicht (Quelle: Eigene Darstellung)

Die wichtigsten Container im aktuellen System sind:

- **Web Application Firewall (WAF)**: Führt IP-Whitelisting sowie Zertifikatsprüfungen durch und schützt die Schnittstellen gegenüber dem Internet.
- **API-Management**: Verwaltet eingehende API-Anfragen, prüft auf Richtlinien (Policies) und leitet validierte Anfragen an die zuständigen Systeme weiter.
- **Web Access Management (WAM)**: Übernimmt die zentrale Authentifizierung (SSO) und prüft anhand der IDP-Nummer im LDAP, ob ein Benutzer autorisiert ist.
- **OpenLDAP**: Verzeichnisdienst zur Speicherung von Benutzerdaten, insbesondere E-Mail-Adressen und IDP-Nummern.
- **EcoHub Provisioning Service**: Eine Spring Boot-Anwendung, bereitgestellt auf Azure Kubernetes (AKS), welche Benutzer über SOAP-Schnittstellen verwalten kann (hinzufügen, aktualisieren, löschen).

- **PAX Portal:** Die Zielplattform, auf die Broker nach erfolgreichem Login weitergeleitet werden, wenn alle Prüfungen bestanden wurden.

Die Kommunikation zwischen diesen Containern erfolgt aktuell sequenziell, mit mehreren Prüfungen und Sicherheitsbarrieren. Änderungen oder Fehler in einem der Container können sich direkt auf das Verhalten des Gesamtsystems auswirken.

Komponentendiagramm (C3) - EcoHub Provisioning Service

Das nachfolgende C3-Diagramm zeigt den internen Aufbau des **EcoHub Provisioning Service** auf Komponentenebene. Der Service ist eine Spring Boot-Anwendung. Er stellt eine SOAP-Schnittstelle zur Verwaltung von Broker-Benutzern bereit. Die Benutzerverwaltung erfolgt durch Interaktion mit einem extern angebundenen OpenLDAP-Verzeichnisdienst.

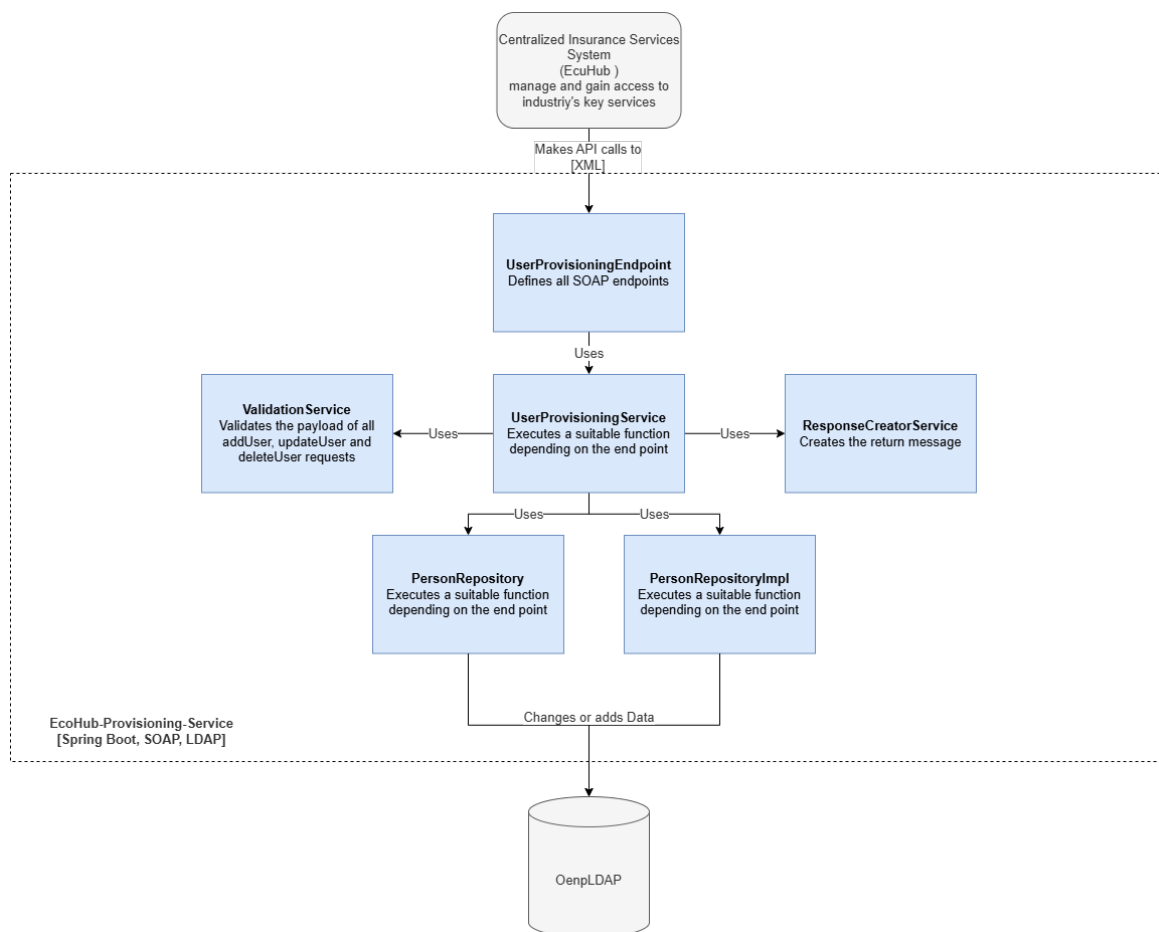


Abbildung 9.5: C3-Komponentendiagramm – EcoHub Provisioning Service (Quelle: Eigene Darstellung)

Die wichtigsten Komponenten im Diagramm sind:

- **UserProvisioningEndpoint**

SOAP-Webservice-Endpunkt, der externe Anfragen wie `addUser`, `updateUser`, `deleteUser` oder `getUserStatus` entgegennimmt.

- **UserProvisioningService**

Zentrale Geschäftslogik. Validiert die Anfragen, delegiert LDAP-Operationen und erstellt Rückgabemeldungen.

- **ValidationService**

Führt strukturierte Prüfungen der eingehenden Payloads durch (z. B. Pflichtfelder, Formate, Längenbeschränkungen) und verwendet Adapter-Objekte zur Entkopplung vom SOAP-Modell.

- **ResponseCreatorService**

Baut standardisierte Antwortobjekte (inkl. Fehlermeldungen) zur Rückgabe über SOAP.

- **PersonRepository & PersonRepositoryImpl**

Repositories zur Suche, Änderung und Löschung von Nutzerdaten im OpenLDAP-Verzeichnis. Nutzen `LdapTemplate` von Spring.

- **OpenLDAP (extern)**

Verzeichnisdienst, der Brokerdaten und ihre Rollen speichert. Wird durch den Provisioning Service gelesen und beschrieben.

- **EcoHub (extern)**

Die EcoHub-Plattform ruft den Provisioning-Service über eine definierte WSDL-Schnittstelle auf und sendet XML-basierte Requests.

Die Kommunikation erfolgt synchron via SOAP, die Daten werden direkt im LDAP-Verzeichnis persistiert. Alle Komponenten sind lose gekoppelt und durch Interfaces und Adapter abstrahiert, was Testbarkeit und Erweiterbarkeit verbessert.

9.1.3 Problemanalyse

Schwachstellen sind konkrete Mängel oder Probleme im aktuellen System oder Prozess. Sie beschreiben, was heute nicht gut funktioniert und somit den Änderungsbedarf begründet. Darunter fallen aktuell folgende Punkte:

- Der Benutzer muss manuell im LDAP eingetragen werden, was ein hoher Aufwand ist und die manuelle Bearbeitung birgt ein hohes Fehlerrisiko.
- Die Rollen von EcoHub werden nicht im LDAP übernommen, was bedeuten kann, dass Broker im EcoHub andere Berechtigungen haben.
- Es gibt keine Tests auf die Endpunkte des EcoHub-Services. Schlägt ein Endpunkt fehl oder werden Änderungen gemacht, wird der Nutzer es als erstes bemerken.

9.1.4 Chancen und Risiken

Risiken sind mögliche Probleme oder Gefahren, die beim Projektverlauf oder der Umsetzung der Änderung auftreten könnten - also zukünftige Unsicherheiten. Folgende wurden für dieses Projekt erkannt.

- Fehlerhafte Zuordnung von Rollen. Die Mappingfunktion weist den Broker in falsche Gruppen und Rollen.
- Änderung an LDAP-Struktur. Ungründliche Vorbereitung und Arbeiten könnte die LDAP-Struktur verändern.
- Unvollständige Testabdeckung. Werden die Funktionen oder Endpunkte nicht vollständig abgedeckt, kann es weiterhin zu unentdeckten Problemen führen.

9.1.5 Zielsetzung

Nun sind verschiedene Ziele für das EcoHub vorgesehen. Der ganze Prozess soll automatisiert geschehen, das bedeutet die manuelle Erstellung der Broker über die Maske fällt weg. Der Broker soll die Möglichkeit bekommen, über das EcoHub sich bei der Pax anzumelden, insofern er eine Service-Vereinbarung eingegangen ist, und damit ein neuer Nutzer im LDAP erstellt und gespeichert wird. Bei der Erstellung wie aber auch nachträglich sollen die Berechtigungen, die der Broker auf der EcoHub-Plattform erhält, die gleichen sein wie die im LDAP. Auch die einzelnen Schnittstellen des EcoHub-Services sollen getestet werden, um Probleme vorzeitig zu erkennen und zu lösen.

9.1.6 Abgrenzung

Wichtig ist, sich nicht in der eigenen Arbeit zu verlieren oder sich in andere Aufgaben zu verlieren. Die Automatisierung des Prozesses, um einen neuen Broker hinzuzufügen, ist nicht Teil der IPA. Auch dass die Gruppen-Zuteilung bei der initialen Anmeldung geschieht, ist nicht Teil der IPA.

9.2 Erweiterte Aufgabenstellung

Die Aufgabe in dieser IPA bezieht sich auf zwei der Ziele. Es soll eine Mappingfunktion geschrieben werden welche die EcoHub Rollen mit den internen Rollen verbindet. Das soll geschehen sollten sich die Rollen des Broker ändern und ein update nötig sein. Auch sollen alle Schnittstellen des EcoHub-Services automatisiert getestet werden, die Automation soll über eine Pipeline laufen.

9.3 Anforderungen

9.3.1 Funktionale Anforderungen

Bei Funktionalen Anforderungen handelt es sich um das (WAS?). Was soll das System oder Produkt tun, besser gesagt welche Funktionen oder Verhaltensweisen muss es bieten. Folgend die wichtigsten Funktionalen

Anforderung im bezug auf die zwei Aufträge.

Mapping-Funktion

- Der Broker wird entsprechend seiner EcoHub-Rolle den korrekten Gruppen im LDAP zugeordnet.
- Gruppen, in denen der Broker nicht mehr Bestandteil ist, aus denen wird er entfernt.
- Bei nicht bekannten Rollen wird die Anfrage ignoriert und der Broker kriegt keine neuen Berechtigungen.
- Die Funktion muss das Mapping aus einer externen Konfigurationsdatei (z.B. application.yml) laden, damit Änderungen ohne Codeanpassung möglich sind.
- Die Funktion muss mehrere Rollen gleichzeitig verarbeiten können.

API-Testautomatisierung

- Alle bekannten API-Endpunkte des EcoHub-Provisioning-Services müssen durch automatisierte Tests abgedeckt werden.
- Die Tests müssen sicherstellen das konkrete Fehlermeldungen zurückgegeben werden.
- Tests müssen jederzeit wiederholbar sein, ohne manuelle Vorbereitung (z.B. Setup- und Teardown-Routinen für Testdaten).
- Die Tests müssen nicht nur wiederholbar sondern auch automatisch durchgeführt werden.

9.3.2 Nicht-funktionale Anforderungen

Bei nicht-funktionalen Anforderungen handelt es sich um das (WIE?). Wie soll das System funktionieren und welche Qualität soll dieses bieten.

Mapping-Funktion

- Die Verarbeitung einer Mapping-Anfrage soll maximal 500 ms dauern.
- Ungültige oder unbekannte Rollen verursachen einen kontrollierten Fehler mit Logging.
- Die Mapping-Logik muss durch Unit-Tests abdeckbar sein.

API-Testautomatisierung

- Die komplette Testausführung soll in unter 2 Minuten durchführbar sein, um schnelle Feedback-Zyklen zu ermöglichen.
- Für die Tests dürfen keine sensitiven Daten verwendet werden. Es werden Testdaten genutzt, welche nicht mit echten Nutzern in Verbindung gebracht werden können.

- Die Testfälle müssen gut strukturiert, benannt und dokumentiert sein, sodass sie auch von Dritten gepflegt werden können.
- Das Testsystem soll erweiterbar sein, damit neue Endpunkte mit geringem Aufwand integriert werden können.

9.4 Rahmenbedingungen und Annahmen

Rahmenbedingungen beschreiben äussere Faktoren, die das Projekt nicht aktiv beeinflussen, an die sich aber gehalten werden muss. Sie definieren sozusagen den Spielraum innerhalb des Projekts. Folgend sind die Rahmenbedingungen dieser Arbeit:

1. Das bestehende LDAP-System (OpenLDAP) darf nicht strukturell verändert werden (Schema, Gruppenstruktur bleibt gleich).
2. Es muss eine bestehende Spring Boot Applikation verwendet werden.
3. Es dürfen nur bereits vorhandene Gruppen im LDAP verwendet werden.
4. Die Integration erfolgt via SOAP-Schnittstelle des EcoHub.
5. Die API-Tests werden mit Playwright gemacht, da Pax weit alle API- und End-to-End-Tests mit Playwright geschrieben werden.

KAPITEL 10

PLANUNG/DESIGN

Gemäss dem Wasserfallmodell folgt auf die Analysephase die Designphase, in der die geplanten Systemanpassungen detailliert ausgearbeitet werden. Aufbauend auf den zuvor identifizierten Schwächen wird in diesem Kapitel beschrieben, wie die Benutzerverwaltung des EcoHub Provisioning Service funktional erweitert und technisch abgesichert werden soll.

Neben der Erweiterung bestehender Funktionalität steht auch die Einführung automatisierter Tests zur Qualitätssicherung im Fokus. Die folgenden Abschnitte stellen das Zielbild, die geplanten Änderungen auf Komponentenebene sowie die vorgesehenen Massnahmen zur Absicherung und Testbarkeit der Erweiterung vor.

10.1 Zielbild und Anforderungen

Im Rahmen der zweiten Projektphase werden zwei zentrale Erweiterungen am EcoHub Provisioning Service vorgenommen:

1. Die Funktion `updateUser` wird um eine automatische Gruppenzuweisung im LDAP erweitert. Die Zuweisung basiert auf den übermittelten Rolleninformationen (`igRoles`) und erfolgt konfigurierbar über ein YAML-basiertes Mapping.
2. Für alle vier vorhandenen SOAP-Endpunkte (`addUser`, `updateUser`, `deleteUser`, `getUserStatus`) werden API-Tests mit Playwright implementiert. Diese Tests werden in die bestehende CI/CD-Pipeline integriert, um eine frühzeitige Fehlererkennung zu ermöglichen.

Die Erweiterungen verfolgen das Ziel, die Benutzerverwaltung zu automatisieren und gleichzeitig die Qualitätssicherung durch systematische Tests zu erhöhen. Dabei wird die bestehende Architektur bewusst nur minimal angepasst und auf Wiederverwendbarkeit sowie Konfigurierbarkeit geachtet.

10.2 Komponentendesign: Erweiterung von updateUser

10.3 Konfigurationsdesign: YAML-Mapping

10.4 Ablaufbeschreibung der Funktion updateUser

10.5 Testdesign und API-Abdeckung

Zur Sicherstellung der Funktionalität und zur frühzeitigen Fehlererkennung werden für alle vier vorhandenen SOAP-Endpunkte systematische API-Tests eingeführt. Die Tests werden mit dem Framework **Playwright** entwickelt, welches durch seine Flexibilität sowohl für Web- als auch für API-Tests geeignet ist.

Zielsetzung

- Automatisierte Tests für: addUser, updateUser, deleteUser, getUserStatus
- Prüfung auf korrekte Verarbeitung valider Payloads
- Erkennung von Fehlerfällen bei ungültigen oder unvollständigen Eingaben
- Integration in die bestehende GitHub-Actions-Pipeline

Testfälle (Auswahl)

Tabelle 10.1: Übersicht der Testfälle

Name	Beschreibung	Input	Erwarteter Output	Tatsächlicher Output
addUser	Nutzer erfolgreich angelegt mit IDP-Nummer	E-Mail: test@example.com	Status: 200, IDP-Nummer generiert	Status: 200, IDP-Nummer: 12345
updateUser	Nutzer aktualisiert und Gruppen gemappt	E-Mail: test@example.com, Rollen: [EL, KL]	Status: 200, Gruppen aktualisiert	Status: 200, Gruppen: [EL, KL]
deleteUser	Nutzer aus LDAP entfernt	E-Mail: test@example.com	Status: 200, Nutzer gelöscht	Status: 200, Nutzer gelöscht
getUserStatus	Existenzprüfung korrekt durchgeführt	E-Mail: test@example.com	Status: 200, Existenz: true	Status: 200, Existenz: true
Fehlerfall	Payload ohne Pflichtfeld	E-Mail: test@example.com (ohne userId)	Status: 400, Fehlercode: MISSING_FIELD	Status: 400, Fehlercode: MISSING_FIELD

Die Tests werden im gleichen Repository wie die Anwendung selbst gehalten. Es wird ein separater Ordner für die Tests erstellt. In der CI/CD-Pipeline wird ein dedizierter Job integriert, der die Tests bei jedem Build ausführt. So können fehlerhafte Implementierungen frühzeitig erkannt und rückgemeldet werden.

10.6 Backupkonzept & Versionierung

Backupkonzept und eine sichere Versionierungsverwaltung ist besonders wichtig. Sollten Daten

10.6.1 Backupkonzept

Während dieser Arbeit wird die 3-2-1-1-0-Methode verwendet. Diese ist eine moderne Erweiterung der klassischen 3-2-1-Methode. In der 3-2-1-Methode stehen die einzelnen Zahlen für folgende Regeln.

3. Mindestens drei Kopien
2. Auf mindestens zwei verschiedenen Medientypen. (z.B. Lokale Festplatte und Cloud)
1. Und mindestens eine Kopie muss an einem geografisch anderen Standort sein.

Das Ziel dieser Methode ist es, einerseits eine sichere Datenenerhältlichkeit zu gewährleisten und ebenfalls bei Problemen keine Daten zu verlieren.

In der modernen 3-2-1-1-0-Methode werden zwei neue Regeln hinzugefügt.

1. Mindestens eine Kopie sollte offline sein, also nicht mit dem Netzwerk verbunden und unveränderlich.
0. Die Kopien dürfen keine Fehler aufweisen und werden dafür regelmässig getestet.

Mit den neuen zwei Regeln werden die Daten nun auch gegen Ransomware oder menschliche Fehler geschützt. Auch das Testen der Backups wird gefordert, um ungewünschte Überraschungen zu vermeiden.

10.6.2 Versionierung

In dieser Arbeit wird für die Versionierung Git mit der Online-Plattform Github verwendet.

Git ist ein verteiltes Versionskontrollsystem, mit dem Entwickler den Verlauf von Änderungen an Dateien insbesondere Quellcode nachverfolgen können. Es ermöglicht mehrere parallele Entwicklungszweige, erleichtert das Zusammenführen von Änderungen und unterstützt das Arbeiten im Team ohne Datenverlust.

GitHub ist die dazu passende Online-Plattform, die Git-Repositories hostet. Sie bietet zusätzliche Funktionen wie Benutzeroberfläche, Pull Requests, Issue-Tracking und CI/CD-Integration. GitHub erleichtert die Zusammenarbeit an Softwareprojekten und macht den Code öffentlich oder privat zugänglich.

KAPITEL 11

IMPLEMENTIERUNG

Beschreibung der Implementierungsphase.

11.1 Mapping-Funktionen

Beschreibung der Mapping-Funktionen.

11.2 Testautomation

Beschreibung der Testautomation.

KAPITEL 12

TEST/QUALITÄTSSICHERUNG

Beschreibung der Test- und Qualitätssicherungsphase.

12.1 Mapping-Funktionen

Beschreibung der Mapping-Funktionen.

12.2 Testautomation

Beschreibung der Testautomation.

KAPITEL 13

ERGEBNISSE

Beschreibung der erzielten Ergebnisse.

13.1 Mapping-Funktionen

Beschreibung der Mapping-Funktionen.

13.2 Testautomation

Beschreibung der Testautomation.

KAPITEL 14

DISKUSSION

Diskussion der Ergebnisse und Erkenntnisse.

14.1 Vergleiche

Beschreibung der Vergleiche.

Teil III

Anhang

HILFSMITTELVERZEICHNIS

- Hilfsmittel 1
- Hilfsmittel 2

ABKÜRZUNGSVERZEICHNIS

IPA Individuelle praktische Arbeit

GLOSSAR

- Begriff 1: Definition
- Begriff 2: Definition

ABBILDUNGSVERZEICHNIS

Abbildungsverzeichnis

2.1	Wasserfallmodell (Quelle: Eigenerarbeitung)	8
3.1	Projektaufbauorganisation (Quelle: Eigenerarbeitung)	11
9.1	Rollenmodell auf der EcoHub-Plattform (Quelle: Miro Board)	21
9.2	Business Process Model des IST-Zustands der Benutzerregistrierung (Quelle: Eigene Darstellung)	21
9.3	Systemkontext (C1) – IST-Zustand der Benutzerverwaltung (Quelle: Eigene Darstellung)	22
9.4	C2-Diagramm IST-Zustand – Containerübersicht (Quelle: Eigene Darstellung)	23
9.5	C3-Komponentendiagramm – EcoHub Provisioning Service (Quelle: Eigene Darstellung)	24

TABELLENVERZEICHNIS

Tabellenverzeichnis

2.1	Vergleich zwischen Wasserfall und agilen Methoden	10
4.1	Aufgabenübersicht	12
9.1	Übersicht der zu testenden Endpunkte	22
10.1	Übersicht der Testfälle	30

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



QUELLENVERZEICHNIS